

高性能结构化图形引擎 用户手册

High-Performance Structured Figure Engine (FigEngine) User Manual

吾买尔江·吾布力卡斯木 张树磊
Omarjan Obulkasim Shulei Zhang

(Version 1.0.0)

2026-03

1. 引言

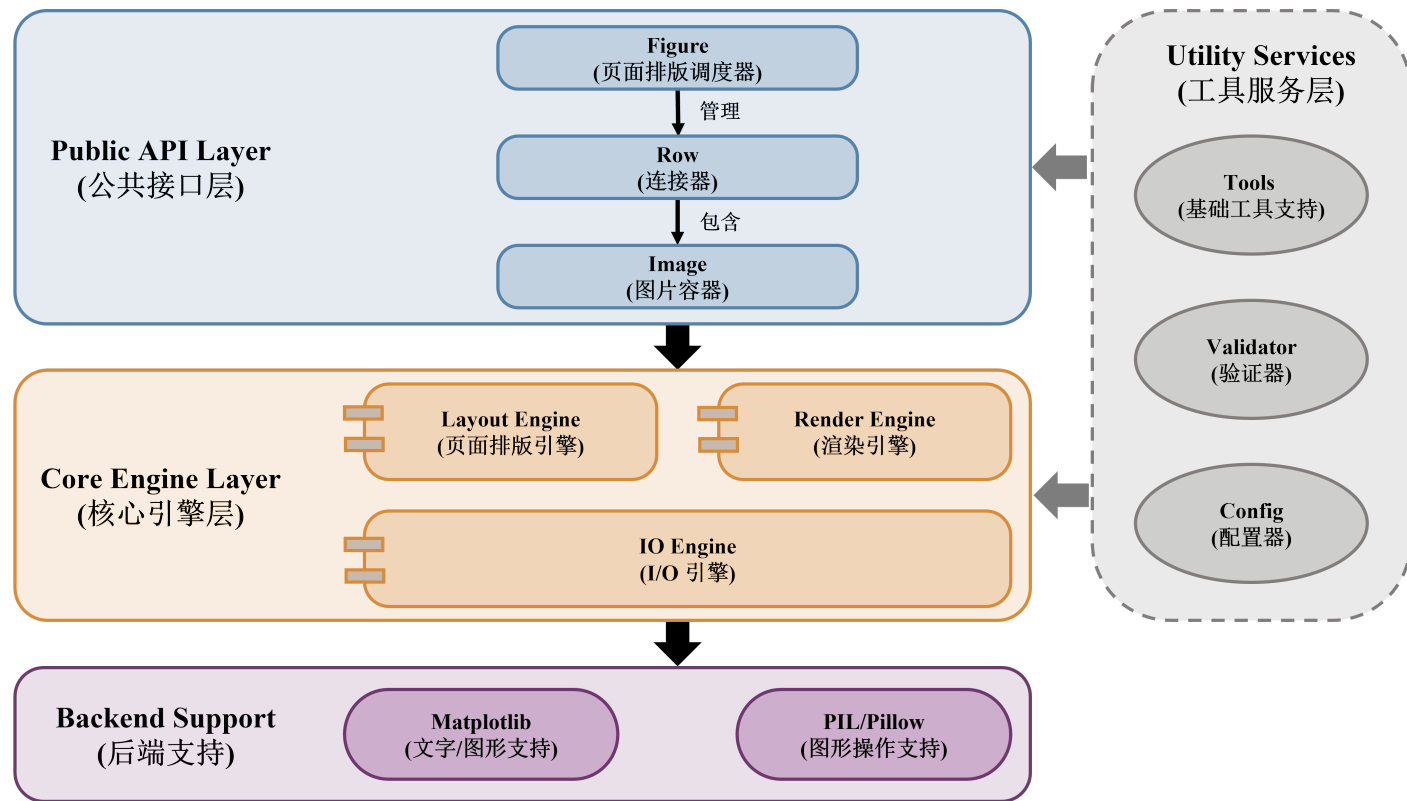
科研工作中，科研人员经常需要将多个实验结果或数据可视化图像进行拼接和排列，生成符合期刊要求的标准图形（Figure）。然而，传统的工具（如 PowerPoint 或 Illustrator）往往需要手动调整图像的大小、位置和对齐方式，这不仅繁琐且容易出错，尤其在面对高分辨率输出时，图像的质量和尺寸控制问题更加突出。FigEngine 的开发正是为了解决这些问题，它通过引入物理单位（如英寸、厘米）支持，帮助科研人员精确控制图像尺寸和位置，同时支持智能化布局和自动化排版，节省了大量手动调整的时间和精力。

与常见的图像处理工具（如 Matplotlib 和 PIL）相比，FigEngine 提供了独特的优势：

- 智能流式布局：用户只需要通过简单的 `add_row()` 调用来添加图像，系统会自动计算布局和间距，避免了手动调整坐标的繁琐过程。这对于多张图像的拼接和排列尤为重要。
- 物理单位支持：引擎原生支持 `inch`、`cm`、`mm` 等物理单位，可以直接在这些单位下进行布局调整，无需转换为像素单位。这使得科研人员能够更精确地满足期刊对图像尺寸的要求。
- 高质量输出：FigEngine 内置智能重采样策略（如 Lanczos 和 Bicubic 插值），确保即使在图像缩放的过程中，依然能保持高质量的输出，避免了常见的模糊或锯齿问题。
- 跨平台一致性：通过分离“排版逻辑”和“渲染逻辑”，FigEngine 能够在不同操作系统和环境下生成一致的图像输出，确保科研成果的可复现性和可对比性。
- 丰富的图像处理支持：提供了强大的图像编辑功能，包括裁剪、缩放、添加标注、绘制几何形状和插入文本，使得用户能够轻松地对图像进行精准调整和美化，满足科研图像的高标准要求。

FigEngine 架构

FigEngine: A High-Performance Structured Scientific Figure Engine (高性能结构化科研绘图引擎)



2. 系统环境要求

FigEngine 是一个高性能的科研图形引擎，旨在为科研工作者提供便捷的图像处理与排版解决方案。

2.1. 软件要求

- 操作系统:
 - Windows 10 或更高版本
 - macOS 10.14 或更高版本
 - Linux (推荐使用 Ubuntu 18.04 或更高版本)
- Python 版本:
 - Python 3.9 或更高版本
- 依赖库:
 - Matplotlib: 用于高质量的图形绘制。

- Pillow (PIL): 用于图像处理和格式转换。
- NumPy: 用于数值计算和数据处理。

2.2. 安装要求

- Python 包管理工具:
 - **pip**: 用于安装和管理 Python 包。
 - **conda** (可选): 如果使用 Anaconda 环境, 建议使用 conda 安装 Python 和依赖库。

2.3. 推荐工具

- **Jupyter Notebook**: 推荐使用 Jupyter Notebook 或 JupyterLab 进行交互式编程和图像展示, 方便实时预览和调整图像。

2.4. 网络连接

- **GitHub**: 如果从 GitHub 安装 FigEngine, 需要一个有效的网络连接来访问远程仓库并下载最新版本。
通过确保符合以上环境要求, 您将能够顺利安装和运行 FigEngine, 并充分利用其功能进行科研图像的高效生成和排版。

3. 安装与部署

FigEngine 提供了多种安装方式, 用户可以根据自己的需求选择合适的安装方法。以下是几种常见的安装方式:

3.1. 从 PyPI 安装 (推荐)

如果您希望直接从 Python Package Index (PyPI) 安装 FigEngine, 只需运行以下命令:

```
1 | pip install figengine
```

这种方式将安装稳定版本的 FigEngine, 并自动处理所有依赖项。

3.2. 从 Conda 安装 (推荐)

如果您希望直接从 Conda 安装 FigEngine, 只需运行以下命令:

```
1 | conda install figengine
```

这种方式将安装稳定版本的 FigEngine, 并自动处理所有依赖项。

3.3. 从 GitHub 安装最新开发版

如果您希望安装 FigEngine 的最新开发版本, 可以从 GitHub 仓库获取。运行以下命令来安装最新的代码版本:

```
1 | pip install git+https://github.com/wubulks/FigEngine.git
```

这将从 GitHub 仓库直接安装最新的开发版, 适合需要体验最新功能或进行开发的用户。

3.4. 从源码安装

如果您希望从源码进行安装，或者需要进行自定义修改，可以按照以下步骤操作：

克隆仓库：

首先，克隆 FigEngine 仓库到本地：

```
1 | git clone https://github.com/wubulks/FigEngine.git
```

进入项目目录：

进入克隆下来的项目目录：

```
1 | cd FigEngine
```

安装依赖：

使用 pip 安装项目依赖：

```
1 | pip install -r requirements.txt
```

安装 FigEngine：

最后，使用以下命令安装 FigEngine：

```
1 | pip install .
```

这样，您就可以从源码安装并运行 FigEngine。

3.5. 安装开发依赖

如果您希望参与开发或修改 FigEngine，可以安装开发依赖：

```
1 | pip install -e .[dev]
```

这将安装所有开发所需的工具，如测试框架、代码质量检查工具等。

3.6. 更新到最新版本

如果已经安装了 FigEngine 并希望更新到最新版本，可以运行以下命令：

```
1 | pip install --upgrade figengine
```

或者，如果您是从 GitHub 安装的开发版，可以使用：

```
1 | pip install --upgrade git+https://github.com/wubulks/FigEngine.git
```

3.7. 验证安装

安装完成后，您可以通过以下命令验证 FigEngine 是否成功安装：

```
import figengine
print(figengine.__version__)
```

如果没有报错并且输出了版本号，说明安装成功。

4. 功能详解

4.1. 图片操作

4.1.1 导入库

```
import os
import sys
# 导入FigEngine库
import figengine as fe
```

4.1.2 可打印的基础信息

```
# 打印版本
print(f"version: {fe.__version__}")
# 打印作者
print(f"author: {fe.__author__}")
# 打印作者邮箱
print(f"author_email: {fe.__author_email__}")
# 打印证书
print(f"license: {fe.__license__}")
# 打印可用字体
fe.Tools.print_valid_fonts()
```

4.1.3 读取图片

功能说明：

该功能用于从文件加载图片并转换为 `Image` 对象，可用于进一步处理、显示等操作。

函数签名：

```
fe.Image(source, label, dpi)
```

参数：

- `source`: 文件路径或已经创建的Image对象。(支持: `.jpg`, `.png`, `.pdf`, `.tif`)
- `label`: 图像标签, 后续可用于生成子图编号
- `dpi`: 图像分辨率(默认 300), 如果 `source` 是文件且包含 DPI 信息, 则自动读取

使用案例：

```
# 从文件加载一个Image对象
img1 = fe.Image(source="assets/img1_rotated_neg25.png", label="Test Image from File", dpi=600)
# 显示图片, 仅建议在jupyter notebook中使用
img1.show(width=500)
```

4.1.4 创建空白填色的Image对象 (new)

功能说明：

该功能允许用户创建一个指定尺寸和背景色的空白图像。

函数签名:

```
fe.Image.new(size, facecolor, unit, dpi, label)
```

参数:

- **size**: 图像尺寸 (width, height)
- **facecolor**: 背景颜色(默认白色)
- **unit**: 尺寸单位(支持: "pixel", "inch", "cm", "mm", 默认 "inch")
- **dpi**: 图像分辨率(默认 600)
- **label**: 图像标签, 后续可用于生成子图编号

使用案例:

```
# 创建一个纯白色的空白图片(6英寸*5英寸)
img1 = fe.Image.new(size=(6.0, 5.0), facecolor="#FFFFFF", unit="inch", dpi=600, label="Test Image")
```

4.1.5 图片属性 (size, dpi, label)

尺寸:

- **Image.size**: 图片尺寸(以像素为单位)
- **Image.get_size(unit)**: 图片尺寸, (unit支持: "pixel", "inch", "cm", "mm", 默认 "pixel")

分辨率:

- **Image.dpi**: 图片分辨率

标签:

- **Image.label**: 图像标签

使用案例:

```
print(f"img1.size: {img1.size}")
print(f"img1.get_size(pixel): {img1.get_size('pixel')}") # 打印像素尺寸
print(f"img1.get_size(inch): {img1.get_size('inch')}") # 打印英寸尺寸
print(f"img1.get_size(cm): {img1.get_size('cm')}") # 打印厘米尺寸
print(f"img1.get_size(mm): {img1.get_size('mm')}") # 打印毫米尺寸
print(f"img1.dpi: {img1.dpi}") # 打印DPI值
print(f"img1.label: {img1.label}") # 打印图像模式
```

4.1.6 添加图片标尺 (add_ticks)

图片位置定位器, 强烈推荐只使用ratio模式, 对于其他功能也是有ratio尽量使用ratio

功能说明:

该功能在图像中添加刻度线, 帮助科研人员在图像中标注坐标轴刻度或其他重要的尺寸参考。

函数签名:

```
Image.add_ticks(step, unit, color, font, fontsize, show_grid)
```

参数:

- **step**: 刻度步长
- **unit**: 坐标单位(支持: "pixel", "ratio", "inch", "cm", "mm", 默认 "ratio")
- **color**: 线条和文字颜色
- **font**: 字体名称, 默认"sans-serif"
- **fontsize**: 字体大小, 默认6。
 - 小于1, 则认为是相对于图片的比例。
 - 大于1, 则认为是字号 (于Matplotlib相同)
- **show_grid**: 是否显示内部网格线, False只在边缘显示刻度 (默认 True)

使用案例:

```
img2 = img1.add_ticks(step=0.01, unit='ratio', color='black', fontsize=0.008)
```

4.1.7 添加文本 (add_text)

功能说明：
该功能用于在图像中添加文本，支持语义化定位和绝对坐标定位，可用于标注、标题、说明等。

- 1. 语义化定位 (Semantic Mode): 不传 x, y
使用 position (e.g. "top_left") + offset (内边距) 来定位
- 2. 绝对坐标 (Absolute Mode): 传 x, y
使用 anchor (e.g. "center") 决定文字如何对齐到 (x, y)

函数签名：
`Image.add_text(text, x, y, loc, anchor, offset, unit, font, fontsize, fontweight, rotation, color, box_style, dpi)`

- 参数：
- **text**: 文本内容, 支持部分latex
 - **x**: 目标位置x (绝对坐标模式)
 - **y**: 标位置y(绝对坐标模式)
 - **loc**: 语义化位置, 不传入x, y时生效(支持: "center", "top", "bottom", "left", "right" 以及复合位置, 如: top_left, 默认"top_left")
 - **anchor**: 文字自身的锚点 (支持: "center", "top", "bottom", "left", "right" 以及复合位置, 如: top_left, 在绝对坐标模式下默认center, 在语义化定位模式下默认与 **loc** 相同)
 - **offset**: 内边距
 - **unit**: 坐标/Offset 的单位(支持: "pixel", "ratio", "inch", "cm", "mm", 默认 "ratio")
 - **fontsize**: 字体大小
 - int: 像素值 (e.g. 24)
 - float < 1.0: 相对高度比例 (e.g. 0.05 = 5% image height)
 - **fontweight**: 字重, 默认"normal", 支持常见的所有类型
 - **rotation**: 文字旋转角度, 默认为0
 - **color**: 文本颜色, 默认为"black"
 - **font**: 字体名称, 默认"sans-serif"
 - **box_style**: (可选) 文本边框样式, 需要传入字典 (例如: {'facecolor': 'white', 'edgecolor': 'red', 'boxstyle': 'round,pad=0.5,rounding_size=0.2'}), 与matplotlib相同
 - **dpi**: (可选) 渲染时的目标 DPI, 默认与图片dpi相同

使用案例：

```
# 语义化定位
img1=img1.add_text("Top Left", loc="top_right", anchor="top_right", offset=0.25,
                    unit="ratio", fontsize=25, color="red")

# 绝对坐标定位
img1=img1.add_text("Bottom Right", x=0.9, y=0.9, unit="ratio", anchor="bottom_right",
                    fontsize=18, color="green")

# 添加文字外边框
img1=img1.add_text("Bottom Center", x=0.7, y=0.7, unit="ratio", anchor="center",
                    fontsize=18, color="green", box_style={'facecolor': 'white', 'boxstyle': 'round,pad=0.5,rounding_size=0.4'})

# 添加latex格式文本
formula_pi = r"${\frac{1}{\pi}} = \frac{2\sqrt{2}}{\sum_{k=0}^{\infty} \frac{(4k)!(1103+26390k)}{(k!)^4 396^{4k}}}$"
img1=img1.add_text(formula_pi, x=0.3, y=0.4, unit="ratio", anchor="center",
                    fontsize=18, color="green", box_style={'facecolor': 'white', 'boxstyle': 'round,pad=0.5,rounding_size=0.4'})
```

4.1.8 添加智能标注 (labeled)

功能说明：
此功能用于为图像添加子图编号或标题，支持自动编号和格式化，常用于科研图像的标注。

函数签名：
`Image.labeled(Label, loc, offset, format_str, case, fontsize, fontweight, color, font, box_style)`

- 参数：
- **label**: (可选) 强制指定标签内容。如果为 None，则使用 self.label。
 - **loc**: 位置 (默认值: "top_left")。支持的位置包括: "center", "top_left", "top_right", "bottom_left", "bottom_right" 等。
 - **offset**: 距离边缘的内边距。建议使用 tuple (x_off, y_off) 做微调，默认值为 (0.02, 0.02)。
 - **format_str**: 格式化字符串，默认值为 "{i}"，例如 "(a)"、"Fig. {i}" 等。
 - **case**: 大小写转换，默认为 None。支持 "upper" (转换为大写) 和 "lower" (转换为小写)。
 - **fontsize**: 字体大小，通常比普通文本稍大。
 - **fontweight**: 字重，默认为 "bold"。
 - **color**: 文字颜色，默认为 "black"。
 - **font**: 字体，默认为 "sans-serif"。
 - **box_style**: 背景框样式，例如 {'facecolor': 'white', 'alpha': 200} 用于复杂背景。

使用案例:

```
# 1. 使用默认标签添加智能标注
img = img.labeled(label="a", loc="top_left", offset=(0.02, 0.02), fontsize=24, fontweight="bold", color="blue")
# 2. 使用格式化合字符串自动生成带括号的编号
img = img.labeled(label="1", loc="top_right", format_str="({})", fontsize=20, color="green")
# 3. 强制使用大写标签, 位置为左下角
img = img.labeled(label="b", loc="bottom_left", case="upper", fontsize=18, fontweight="normal", color="red")
# 4. 使用默认 `self.label` 和格式 "Fig. {}"
img = img.labeled(loc="top_right", format_str="Fig. {}", fontsize=20, color="black")
# 5. 使用带背景框的标注
img = img.labeled(label="C", loc="bottom_center", fontsize=22, color="purple",
                  box_style={'facecolor': 'white', 'alpha': 0.5, 'boxstyle': 'round,pad=0.5,rounding_size=0.2'})
```

说明:

- 自动标签内容: 如果没有指定 `label`, 则默认使用 `self.label`, 或者如果 `self.label` 也没有定义, 则使用 `"?"`。
- 格式化支持: `format_str` 参数支持用户自定义标签格式。可以选择包括括号、前缀等, 例如使用 `"({})"` 格式化合字符串可以生成类似 (a) 这样的标签。
- 大小写控制: 通过 `case` 参数, 用户可以轻松转换标签内容为大写或小写, 适应不同的图表规范需求。

4.1.9 添加线条 (`add_line`)

功能说明:

此功能用于在图像中绘制线条, 可以指定线条的起止点、颜色、宽度等属性, 支持添加箭头。

函数签名:

```
Image.add_line(start, end, unit, color, width, arrow, arrow_size, arrow_style, arrow_angle, arrow_shorten, arrow_fill)
```

参数:

- `start`: 起点坐标 (`x1`, `y1`)
- `end`: 终点坐标 (`x2`, `y2`)
- `unit`: 坐标单位 (支持: `"pixel"`, `"ratio"`, `"inch"`, `"cm"`, `"mm"`, 默认 `"ratio"`)
- `color`: 线条颜色
- `width`: 线条宽度, 默认 `0.01`
- `arrow`: 箭头位置 (支持: `"start"`, `"end"`, `"both"` 或 `None`, 默认 `None`)
- `arrow_size`: 箭头斜边长度 (像素)
- `arrow_style`: 箭头样式 (支持: `"triangle"`, `"open"`, `"bar"`, `"diamond"`, `"circle"`, 默认: `"triangle"`)
- `arrow_angle`: 箭头夹角(度), 仅 `triangle/open` 有效
- `arrow_shorten`: 主线缩短量, `None` 时默认等于 `arrow_size x 0.5`
- `arrow_fill`: `triangle/diamond` 时是否填充 (`True` =填充; `False` =只描边, 默认填充)

使用案例:

```
img2 = img1.add_line(start=(0.1, 0.1), end=(0.9, 0.1), color="#CAEFD6", width=0.01, arrow="end", arrow_size=0.05, arrow_shorten=0.02,
                    arrow_style="triangle")
```

4.1.10 添加边框 (`add_rect`)

功能说明:

此功能允许用户为图像添加矩形边框, 可以通过起点/终点或中心/尺寸来定义矩形区域。

函数签名:

```
Image.add_rect(start, end, center, size, unit, linewidth, color, edgecolor, facecolor, fill)
```

参数:

模式1: 起始坐标模式

- `start`: 起点坐标 (`x1`, `y1`)
 - `end`: 终点坐标 (`x2`, `y2`)
- 模式2: 中心坐标模式
- `center`: 中心坐标 (`cx`, `cy`)
 - `size`: 中心坐标 (`width`, `height`)

通用参数

- **unit**: 坐标单位,该单位适用于 **start/end** 或 **center/size** 以及 **linewidth**。(支持: "pixel", "ratio", "inch", "cm", "mm", 默认 "ratio")
- **linewidth**: 线条宽度, 默认 0.01
- **color**: 基本颜色, 若未指定 **edgecolor/facecolor** 则以此为准
- **edgecolor**: 边框颜色, 覆盖 **color**。
- **facecolor**: 填充颜色, 覆盖 **color**。
- **fill**: 是否填充(**True** = 填充; **False** = 只描边, 默认不填充)

使用案例:

```
img2 = img1.add_rect(start=(0.1, 0.1), end=(0.9, 0.1), unit="ratio", linewidth=0.01, color="#CAEFD6")
img2 = img1.add_rect(center=(0.5, 0.5), size=(5, 6), unit="inch", linewidth=0.5, color="#CAEFD6")
```

4.1.11 添加圆形/椭圆 (**add_oval**)

功能说明:

此功能用于在图像中添加圆形或椭圆, 可以通过设置中心点、半径和轴比来精确控制形状。

函数签名:

Image.add_rect(start, end, center, radius, axis_ratio, unit, linewidth, color, edgecolor, facecolor, fill)

参数:

模式1: 起始坐标模式

- **start**: 起点坐标 (**x1**, **y1**)。
- **end**: 终点坐标 (**x2**, **y2**)。

模式2: 起始坐标模式

- **center**: 中心坐标 (**cx**, **cy**)。
- **radius**: 短轴半径。
- **axis_ratio**: 长轴/短轴比例, 1.0 表示圆形, 默认为 1.0。

通用参数

- **unit**: 坐标单位,该单位适用于 **start/end** 或 **center/size** 以及 **linewidth**。(支持: "pixel", "ratio", "inch", "cm", "mm", 默认 "ratio")。
- **linewidth**: 线条宽度, 默认 0.01。
- **color**: 基本颜色, 若未指定 **edgecolor/facecolor** 则以此为准。
- **edgecolor**: 边框颜色, 覆盖 **color**。
- **facecolor**: 填充颜色, 覆盖 **color**。
- **fill**: 是否填充(**True** = 填充; **False** = 只描边, 默认填充)。

使用案例:

```
img2 = img1.add_oval(center=(0.5, 0.5), radius=0.3, axis_ratio=1.5, unit="ratio", color="blue", fill=True)
```

4.1.12 添加特征点 (**add_marker**)

功能说明:

此功能用于在图像上添加特征点, 可以选择不同的样式、大小、颜色等。适用于标记特定位置或数据点。

函数签名:

Image.add_marker(x, y, unit, style, size, color, outline, width)

参数:

- **x**: 目标位置x
- **y**: 标位置y
- **style**: 点的样式
 - **circle**: "●" 圆形
 - **square**: "■" 正方形
 - **diamond**: "◆" 菱形
 - **triangle_up**: "▲" 正三角
 - **triangle_down**: "▼" 倒三角
 - **pentagon**: "⬠" 五边形
 - **target**: "◎" 靶心
 - **plus**: "+" 加号
 - **cross**: "×" 叉号

- **size**: 点的大小
- **color**: 填充颜色
- **outline**: 边框颜色 (None 则无边框, 对 plus/cross 无效)
- **width**: 边框或线条宽度

使用案例:

```
# 再图片中心添加红色的圆形
img2 = img1.add_marker(x=0.1, y=0.1, unit="ratio", style="circle", size=0.02, color="red")
```

4.1.13 调整图片大小 (**resize**)

功能说明:

该功能用于调整图像的大小, 可以按指定的宽度、高度、比例或参考图像进行缩放。

函数签名:

```
Image.resize(width, height, scale, ref_image, unit, resample)
```

参数:

- **width**: 图像宽度
- **height**: 图像高度
- **scale**: 缩放比例
- **ref_image**: 参考图片
- **unit**: 尺寸单位 (支持: "pixel", "inch", "cm", "mm", 默认 "pixel")
- **resample**: 插值算法
 - **"auto"**: 智能选择 (放大用 bicubic, 缩小用 lanczos), 默认方案
 - **"lanczos"**: 高质量缩小
 - **"bicubic"**: 平滑放大
 - **"nearest"**: 最近邻 (保硬边)

优先级逻辑:

1. **ref_image**: 若存在, 强制缩放到与参考图完全一致 (忽略其他参数)。

2. **width/height**:

- 若两者都有: 强制拉伸到指定尺寸 (忽略原比例)。
- 若只有一个: 保持原图宽高比, 自动计算另一个维度。
- 支持 **unit** 参数 (pixel, inch, cm, mm)。

2. **width/height**:

- **float**: 整体等比缩放。
- **tuple (sx, sy)**: 宽高分别缩放。

使用案例:

```
# 调整图像大小
print("原始比例", img1.size, img1.get_size("inch"), img1.get_size("cm"))

# 强制指定高度和宽度
img2 = img1.resize(width=4.0, height=2.5, unit="inch")
print("强制指定高度和宽度", img2.size, img1.get_size("inch"), img1.get_size("cm"))

# 仅指定宽度, 高度等比例缩放
img3 = img1.resize(width=4.0, unit="inch")
print("仅指定宽度, 高度等比例缩放", img3.size, img3.get_size("inch"), img3.get_size("cm"))

# 仅指定高度, 宽度等比例缩放
img4 = img1.resize(height=10, unit="cm")
print("仅指定高度, 宽度等比例缩放", img4.size, img4.get_size("inch"), img4.get_size("cm"))

# 按比例缩放50%
img5 = img1.resize(scale=0.5)
print("按比例缩放50%", img5.size, img5.get_size("inch"), img5.get_size("cm"))

# 参照另一张图像的大小进行缩放
img6 = img1.resize(ref_image=img2)
print("参照另一张图像的大小进行缩放", img6.size, img6.get_size("inch"), img6.get_size("cm"))
```

4.1.14 图片裁剪 (crop)

功能说明：

此功能用于裁剪图像，可以通过指定绝对坐标或裁剪量来裁剪图像的区域。

函数签名：

`Image.crop(box, left, top, right, bottom, unit)`

参数：

两种模式

- 模式1: 绝对坐标模式
 - `box`: `box=(x1, y1, x2, y2)`，则代表提取该绝对坐标区域。`box` 优先级最高。
- 模式2: 裁剪量模式
 - `left`: 左侧切除量
 - `top`: 顶部切除量
 - `right`: 右侧切除量
 - `bottom`: 右侧切除量
- 共有参数:
 - `unit`: 单位(支持: "pixel", "ratio", "inch", "cm", "mm", 默认 "pixel")

使用案例：

```
print("原始比例", img1.size)

# 绝对坐标模式
img2 = img1.crop(box=(1.0, 1.0, 5.0, 4.0), unit="inch")
print("绝对坐标模式", img2.size)

# 裁剪量模式
img3 = img1.crop(left=1.0, top=1.0, right=1.0, bottom=1.0, unit="inch")
print("裁剪量模式(英寸)", img3.size)
img4 = img1.crop(left=0.1, top=0.1, right=0.1, bottom=0.1, unit="ratio")
print("裁剪量模式(比例)", img4.size)
```

4.1.15 调整画布尺寸 (pad_to_size)

功能说明：

此功能用于通过扩展画布的背景色来调整图像的尺寸，不会改变图像本身的大小。

函数签名：

`Image.pad_to_size(target_size, unit, loc, axis, bg_color)`

参数：

- `target_size`: 目标尺寸 (w, h)
 - `unit`: 单位(支持: "pixel", "ratio", "inch", "cm", "mm", 默认 "pixel")
 - `loc`: 原图在新画布中的锚点位置 (type: str, default: "center"), e.g. "top_left" 表示原图固定在左上角，向右下方扩展。
 - `axis`: 仅调整某个轴向，
 - "both", "width", "height"
 - `bg_color`: 新增区域的填充背景色，默认透明
- 请注意，`pad_to_size` 只会通过扩展背景色调整画布尺寸，并不会改变图片元素本身的大小

使用案例：

```
# 以比例模式进行填充，将原来的图片扩展尺寸至1.1倍
img2 = img1.pad_to_size(target_size=(1.1, 1.1), unit="ratio", bg_color="#B8FFCD")
print("以比例模式进行填充", img2.size)
```

4.1.16 添加边框 (add_border)

功能说明：

此功能用于为图像添加边框，可以设置边框的宽度、颜色以及是否应用于特定边。

函数签名：

`Image.add_border(thickness, unit, left, top, right, bottom, color)`

参数:

- **thickness**: 目标尺寸 (w, h)
 - **unit**: 单位(支持: "pixel", "ratio", "inch", "cm", "mm", 默认 "ratio")
 - **left**: 左侧切除量
 - **top**: 顶部切除量
 - **right**: 右侧切除量
 - **bottom**: 右侧切除量
 - **color**: 颜色, 默认白色
- 添加边框/白边 (Padding only, 不裁剪、不缩放)。

使用案例:

```
# 添加黑色边框并自定义右边框
img2 = img1.add_border(0.005, unit='ratio', left=0.01, color='#F87SF7')

# 仅在右边添加边框
img2 = img1.add_border(unit='ratio', left=0.01, color='#F87SF7')
```

4.1.17 叠加图片 (overlay)

功能说明:

此功能用于将两张图像进行叠加, 可以调整叠加图像的位置、大小和背景色。

函数签名:

`Image.overlay(other, x, y, anchor, unit, scale, expand, bg_color)`

参数:

- **other**: 要叠加的图片对象
- **x**: 叠加基准点 x
- **y**: 叠加基准点 y
- **anchor**: **other** 图片自身的对齐点
- **unit**: 单位(支持: "pixel", "ratio", "inch", "cm", "mm", 默认 "ratio")
- **scale**: **other** 图片的缩放比例, 默认为1, 不缩放。
- **expand**: 是否扩展画布, 默认 **False**。
 - **False**: 裁剪超出边界的部分 (画中画模式)。
 - **True**: 自动扩大画布以容纳 **other** (拼贴模式)。
- **bg_color**: 颜色, 默认白色

使用案例:

```
# 叠加
img_ = img1.overlay(other=img2, x=0.1, y=0.1, unit="ratio", anchor="center", scale=0.3)
```

4.1.18 图片旋转 (rotate)

功能说明:

该功能用于旋转图像, 可以选择是否扩展画布以适应旋转后的图像。

函数签名:

`Image.rotate(angle, expand, bg_color)`

参数:

- **angle**: 旋转角度 (度)
- **expand**: 是否扩展画布, 默认 **False**。
 - **False**: 裁剪超出边界的部分 (画中画模式)。
 - **True**: 自动扩大画布以容纳 **other** (拼贴模式)。
- **bg_color**: 颜色, 默认白色

使用案例:

```
# 叠加
img4 = img4.rotate(angle=30, expand=False, bg_color="FFFFFF")
```

4.1.19 图片展示 (`show`)

功能说明：

此功能用于在 Jupyter Notebook 中显示图像，可以指定图像的显示宽度和缩放比例。

函数签名：

`Image.show(width, scale)`



Danger

此功能只能基于 `jupyter notebook` 使用

参数：

- `width`: 指定显示的宽度 (像素)。例如 `width=500`。
- `scale`: 指定缩放比例。例如 `scale=0.5` (缩小一半显示)。(注意：`width` 优先级高于 `scale`)。

使用案例：

```
# 叠加
img4 = img4.show(width=500)
```

4.2.20 保存图片

功能说明：

该功能用于保存图像为文件，可以指定保存路径和格式。

函数签名：

`Image.save(path, **kwargs)`

参数：

- `path`: 输出文件路径 (e.g. "output/fig1.png"), 需要带格式后缀
- `**kwargs`: 额外参数, 例如 `quality=95` (JPG), `compression="tiff_lzw"` (TIFF) 等

使用案例：

```
fig.save("./assets/figure_layout_output.png")
fig.save("./assets/figure_layout_output.pdf")
```

4.2. 排版布局

4.2.1 导入库

```
import os
import sys
# 导入FigEngine库
import figengine as fe
```

4.2.2 创建Figure

功能说明：

`Figure` 是 FigEngine 的排版编排器 (Orchestrator)，用于组织多张图片的行列布局、间距、对齐方式与最终输出。它负责“排版逻辑”，并在需要输出时触发渲染 (Lazy Evaluation)。

⚠ 懒加载机制 (Lazy Evaluation)

`Figure` 只有在访问 `fig.image` 或调用 `fig.save()` 时才会真正渲染为位图/文件。

函数签名:

```
fe.Figure(background, dpi, width, height, unit)
```

参数:

- **background**: 画布背景颜色
- **dpi**: 输出图像的分辨率, 默认300
- **width**: (可选) 独立指定画布宽度, 如果为 0, 宽度将根据内容自动推断 (建议优先指定宽度)
- **height**: (可选) 独立指定画布高度, 如果为 0, 高度将根据内容自动推断 (谨慎使用)
- **unit**: 尺寸单位(支持: "pixel", "inch", "cm", "mm", 默认 "inch")

使用案例:

```
fig = fe.Figure(background="#FFFFFF", dpi=DPI, width=12, unit="inch")
fig.set_margins(top=0.01, bottom=0.01, left=0.01, right=0.01)
```

4.2.3 Figure中添加Image

功能说明:

该功能用于向 **Figure** 中添加“一行图片”, 并由引擎根据行内对齐策略与间距参数自动完成排版。
它是 FigEngine 的核心工作流: 不断 **add_row()** 组织多行, 最终输出为一个完整的 **Figure**。

函数签名:

```
Figure.add_row(items, left_gaps, right_gaps, top_margin, bottom_margin, unit, align)
```

参数:

- **items**: 图片列表, 可以是文件路径字符串, 也可以是已加载的 **Image** 对象
- **left_gaps**: 图片左侧间距
 - 单个值: 应用于该行所有图片
 - 列表: 逐个对应每张图片
- **right_gaps**: 图片右侧间距
 - 单个值: 应用于该行所有图片
 - 列表: 逐个对应每张图片
- **top_margin**: 该行距离上一行的间距 (单个值)
- **bottom_margin**: 该行距离下一行的间距 (单个值)
- **unit**: 单位(支持: "pixel", "ratio", "inch", "cm", "mm", 默认 "ratio")
- **v_align**: 行内图片的对齐/排版方式
 - **full**: 强制两端对齐 (Justified), 自动缩放图片以等高并填满行宽
 - **top**: 顶部对齐
 - **center**: 垂直居中对齐
 - **bottom**: 底部对齐
- **h_align**: 行之间的对齐/排版方式, 默认 **full**
 - **full**: 按行宽等比缩放图片 (间距固定)
 - **Justified**: 两端对齐
 - **center**: 顶部对齐
 - **left**: 左对齐
 - **right**: 垂直居中对齐
 - **justify**: 底部对齐

使用案例:

```
# 第一行
images_row1 = {"items": [img1, img2, img3],
               "left_gaps": [0.01, 0.01, 0.01],
               "right_gaps": [0.01, 0.01, 0.01],
               "top_margin": 0.01,
               "bottom_margin": 0.01,
               "unit": "ratio",
               "v_align": "center"}
fig_ = fig.add_row(**images_row1)
# 第二行
images_row2 = {"items": [img4, img5, img6],
               "left_gaps": [0.01, 0.01, 0.01],
               "right_gaps": [0.01, 0.01, 0.01],
               "top_margin": 0.01,
               "bottom_margin": 0.01,
               "unit": "ratio",
```

```
        "v_align": "top"}
fig.add_row(**images_row2)
```

4.2.4 删除某一行

功能说明：

该功能用于从 **Figure** 中删除指定索引的一行，常用于交互式排版调试与布局迭代（例如先搭框架，再删改重排）。

函数签名：

Figure.remove_row(index)

参数：

- **index**: 指定索引处的行, 从0开始

使用案例：

```
# 删除第一行
fig.remove_row(index=0)
fig.image.show(width=500)
```

4.2.5 替换Figure中的某一行

功能说明：

该功能用于将 **Figure** 中某一行整体替换为新的行配置，适用于快速尝试不同的图像组合、间距或对齐策略，而无需重建整个 **Figure**。

函数签名：

Figure.replace_row(index, items, left_gaps, right_gaps, top_margin, bottom_margin, unit, align)

参数：

- **index**: 要替换的行的从 0 开始的索引
- **items**: 图片列表, 可以是文件路径字符串, 也可以是已加载的 **Image** 对象
- **left_gaps**: 图片左侧间距（单个值或列表）
- **right_gaps**: 图片右侧间距（单个值或列表）
- **top_margin**: 该行距离上一行的间距（单个值）
- **bottom_margin**: 该行距离下一行的间距（单个值）
- **unit**: 单位(支持: "pixel", "ratio", "inch", "cm", "mm", 默认 "ratio")
- **h_align/v_align**: 行内图片的对齐/排版方式（同 4.2.3）

使用案例：

```
# 替换第一行为第二行
images_row2 = {"items": [img4, img5, img6],
               "left_gaps": [0.01, 0.01, 0.01],
               "right_gaps": [0.01, 0.01, 0.01],
               "top_margin": 0.01,
               "bottom_margin": 0.01,
               "unit": "ratio",
               "v_align": "top"}
fig.replace_row(0, **images_row2)
```

4.2.6 保存图片

功能说明：

该功能用于将 **Figure** 排版结果渲染并保存到文件（如 PNG/PDF/TIFF）。保存过程会触发懒加载渲染，并输出期刊友好的高分辨率结果。

函数签名：

Figure.save(path, **kwargs)

参数：

- **path**: 输出文件路径 (e.g. "output/fig1.png"), 需要带格式后缀
- ****kwargs**: 额外参数, 例如 quality=95 (JPG), compression="tiff_lzw" (TIFF) 等

使用案例：

```
fig.save("./assets/figure_layout_output.png")
fig.save("./assets/figure_layout_output.pdf")
```